UNIVERSITY OF AZAD JAMMU & KASHMIR

LAB TASK 03

Name : Tayyab Hanif Awan

Roll No: 2024-SE-11

Submitted to: Engr. Fahad Nisar

. Lab Objectives

- Understand grayscale image representation and pixel intensity values.
- Apply brightness and contrast transformations.
- Generate a negative image and perform gamma correction.
- Calculate and visualize an image histogram.
- Improve image contrast using global histogram equalization and CLAHE.
- Convert grayscale images into binary images using fixed, Otsu, and adaptive thresholding.
- Compare the visual results of different enhancement techniques.

2. Software and Libraries Used

- Python 3
- Jupyter Notebook
- NumPy – numerical arrays and pixel calculations
- OpenCV (cv2) – image reading and image processing
- Matplotlib – displaying images and plotting histograms

3. Input Image and Initial Inspection

The input image was saved in the lab working folder with the filename Leisuree.jpg. It was loaded in grayscale mode. A grayscale image normally stores intensity values from 0 (black) to 255 (white) using the uint8 data type.

The shape reports the number of rows and columns. The data type uint8 means that each pixel is stored as an 8-bit unsigned integer. The minimum and maximum values describe the intensity range present in the image.

[3]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("Leisure.jpg", cv2.IMREAD_GRAYSCALE)

if img is not None:
    print("Image loaded successfully!")
    print("Shape:", img.shape)
    print("Data Type:", img.dtype)
    print("Minimum Pixel:", img.min())
    print("Maximum Pixel:", img.max())
else:
    print("Image not found!")
```

```
Image loaded successfully!
Shape: (1536, 1024)
Data Type: uint8
Minimum Pixel: 0
Maximum Pixel: 254
```

[4]:
```
plt.figure(figsize=(6, 6))

plt.imshow(img, cmap="gray", vmin=0, vmax=255)

plt.title("Original Grayscale Image")
plt.axis("off")

plt.show()
```



5. Task 1 – Brightness Transformation

Objective: To increase and decrease the brightness of a grayscale image by adding or subtracting a constant value.

Concept: For brightness adjustment, a constant value is added to or subtracted from every pixel:
s = r + b. Here, r is the original pixel and b is the brightness value. Since uint8 pixels must remain between 0 and 255, values are clipped to this range.

Dry Run / Manual Calculation

Given pixels: [30, 80, 128, 180, 240]. For +40: raw values = [70, 120, 168, 220, 280], after clipping = [70, 120, 168, 220, 255]. For -40: raw values = [-10, 40, 88, 140, 200], after clipping = [0, 40, 88, 140, 200].

Python Implementation

```
pixels = np.array([30, 80, 128, 180, 240], dtype=np.uint8)


bright_raw = pixels.astype(np.int16) + 40
bright = np.clip(bright_raw, 0, 255).astype(np.uint8)


dark_raw = pixels.astype(np.int16) - 40
dark = np.clip(dark_raw, 0, 255).astype(np.uint8)


print("Original:", pixels) print("Bright Raw:",
bright_raw) print("Bright Final:", bright) print("Dark
Raw:", dark_raw) print("Dark Final:", dark)


bright_img = cv2.convertScaleAbs(img, alpha=1, beta=40)
dark_img = np.clip(img.astype(np.int16) - 40, 0, 255).astype(np.uint8)


plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1); plt.imshow(img, cmap="gray"); plt.title("Original"); plt.axis("off") plt.subplot(1, 3, 2); plt.imshow(bright_img,
cmap="gray"); plt.title("Brightness +40"); plt.axis("off")
plt.subplot(1, 3, 3); plt.imshow(dark_img, cmap="gray"); plt.title("Brightness -40"); plt.axis("off") plt.show()
```

Observation

Adding 40 makes the image brighter, while subtracting 40 makes it darker. Clipping prevents pixel values from going below 0 or above 255.



6. Task 2 – Linear Contrast Transformation

Objective: To change image contrast by multiplying pixel values with a constant factor.

Concept: The linear transformation is s = alpha × r + beta. In this experiment, alpha = 1.5 and beta = 0. A value of alpha greater than 1 increases contrast. Results are clipped to 0–255.

Dry Run / Manual Calculation

For r = [30, 80, 128, 180, 240], alpha = 1.5: raw output = [45, 120, 192, 270, 360]. After clipping = [45, 120, 192, 255, 255].

Python Implementation

```
pixels = np.array([30, 80, 128, 180, 240], dtype=np.uint8)
  alpha = 1.5
beta = 0
raw = alpha * pixels.astype(np.float32) + beta result = np.clip(raw, 0,
255).astype(np.uint8)


print("Original:", pixels) print("Raw Output:", raw)
print("Final Output:", result)


contrast_img = cv2.convertScaleAbs(img, alpha=alpha, beta=beta)


plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1); plt.imshow(img, cmap="gray"); plt.title("Original"); plt.axis("off") plt.subplot(1, 2, 2);
plt.imshow(contrast_img, cmap="gray"); plt.title("Linear Contrast alpha=1.5"); plt.axis("off") plt.show()
```

Observation

The contrast factor 1.5 spreads intensity values away from zero. Some high values reach 255 because of clipping.



7. Task 3 – Centered Contrast Transformation

Objective: To increase or decrease contrast while keeping the middle intensity value near 128 as the reference point.

Concept: The transformation is s = (r − 128) × alpha + 128. When alpha is greater than 1, contrast increases. When alpha is between 0 and 1, contrast decreases. Values are clipped to 0–255.

Dry Run / Manual Calculation

For r = [30, 80, 128, 180, 240]: with alpha = 1.5, raw values are approximately [-19, 56, 128, 206, 296], so final values are [0, 56, 128, 206, 255]. With alpha = 0.5, values are [79, 104, 128, 154, 184].

Python Implementation

```
pixels = np.array([30, 80, 128, 180, 240], dtype=np.uint8)
  def centered_contrast(values, alpha):     raw = (values.astype(np.float32) -
128) * alpha + 128     return np.clip(raw, 0, 255).astype(np.uint8)


high_contrast = centered_contrast(pixels, 1.5) low_contrast =
centered_contrast(pixels, 0.5)


print("Original:", pixels) print("Alpha 1.5:",
high_contrast) print("Alpha 0.5:", low_contrast)
  high_img = centered_contrast(img, 1.5) low_img =
centered_contrast(img, 0.5)


plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1); plt.imshow(img, cmap="gray"); plt.title("Original"); plt.axis("off") plt.subplot(1, 3, 2); plt.imshow(high_img,
cmap="gray"); plt.title("Increased Contrast 1.5"); plt.axis("off")
plt.subplot(1, 3, 3); plt.imshow(low_img, cmap="gray"); plt.title("Decreased Contrast 0.5"); plt.axis("off") plt.show()
```

Observation

Alpha = 1.5 increases the difference between dark and bright pixels. Alpha = 0.5 reduces the difference and makes the image look flatter.

8. Task 4 – Image Negative

Objective: To create the negative (inverted intensity) of a grayscale image.

Concept: For an 8-bit grayscale image, the negative transformation is s = 255 − r. Dark pixels become bright and bright pixels become dark.

Dry Run / Manual Calculation

For pixels [30, 80, 128, 180, 240], the negative is [225, 175, 127, 75, 15]. Applying the negative operation twice returns the original values.

Python Implementation

```
pixels = np.array([30, 80, 128, 180, 240], dtype=np.uint8)


negative = 255 - pixels
double_negative = 255 - negative


print("Original:", pixels) print("Negative:", negative)
print("Double Negative:", double_negative)


negative_img = 255 - img


plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1); plt.imshow(img, cmap="gray"); plt.title("Original"); plt.axis("off") plt.subplot(1, 2, 2);
plt.imshow(negative_img, cmap="gray"); plt.title("Image Negative"); plt.axis("off") plt.show()
```

Observation

The negative image reverses brightness. Areas that were dark become bright, and areas that were bright become dark.



9. Task 5 – Gamma Correction

Objective: To adjust image brightness and contrast non-linearly using gamma values.

Concept: The gamma transformation is s = 255 × (r/255)^gamma. Gamma less than 1 brightens an image, while gamma greater than 1 darkens it. The output is converted to uint8 after calculation.

Dry Run / Manual Calculation

For pixels [0, 64, 128, 192, 255]: gamma = 0.5 gives approximately [0, 127, 180, 221, 255]. Gamma = 2 gives [0, 16, 64, 144, 255]. Fractional values are truncated when converted to uint8.

Python Implementation

```
pixels = np.array([0, 64, 128, 192, 255], dtype=np.uint8)
  def gamma_transform(image, gamma):     normalized =
image.astype(np.float32) / 255.0     corrected = 255 * (normalized
** gamma)
   return np.clip(corrected, 0, 255).astype(np.uint8)


print("Original:", pixels)
print("Gamma 0.5:", gamma_transform(pixels, 0.5)) print("Gamma 2:",
gamma_transform(pixels, 2.0))


gamma_bright = gamma_transform(img, 0.5) gamma_dark =
gamma_transform(img, 2.0)


plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1); plt.imshow(img, cmap="gray"); plt.title("Original"); plt.axis("off") plt.subplot(1, 3, 2); plt.imshow(gamma_bright,
cmap="gray"); plt.title("Gamma 0.5"); plt.axis("off")
plt.subplot(1, 3, 3); plt.imshow(gamma_dark, cmap="gray"); plt.title("Gamma 2.0"); plt.axis("off") plt.show()
```

Observation

Gamma 0.5 makes darker regions more visible. Gamma 2.0 reduces intensity in many regions and produces a darker result.



10. Task 6 – Image Histogram

Objective: To calculate and display the frequency of each intensity level in a grayscale image.

Concept: A histogram counts how many pixels have each intensity value from 0 to 255. A grayscale uint8 image therefore has 256 possible intensity bins. The probability of an intensity is its frequency divided by the total number of pixels.

Dry Run / Manual Calculation

For the 2×4 array [[0, 50, 50, 100], [100, 150, 200, 255]], the frequencies are: 0:1, 50:2, 100:2, 150:1, 200:1, 255:1. Total pixels = 8. For example, probability of intensity 50 = 2/8 = 0.25. All intensity probabilities add up to 1.

Python Implementation

```
sample = np.array([[0, 50, 50, 100],
                   [100, 150, 200, 255]], dtype=np.uint8)

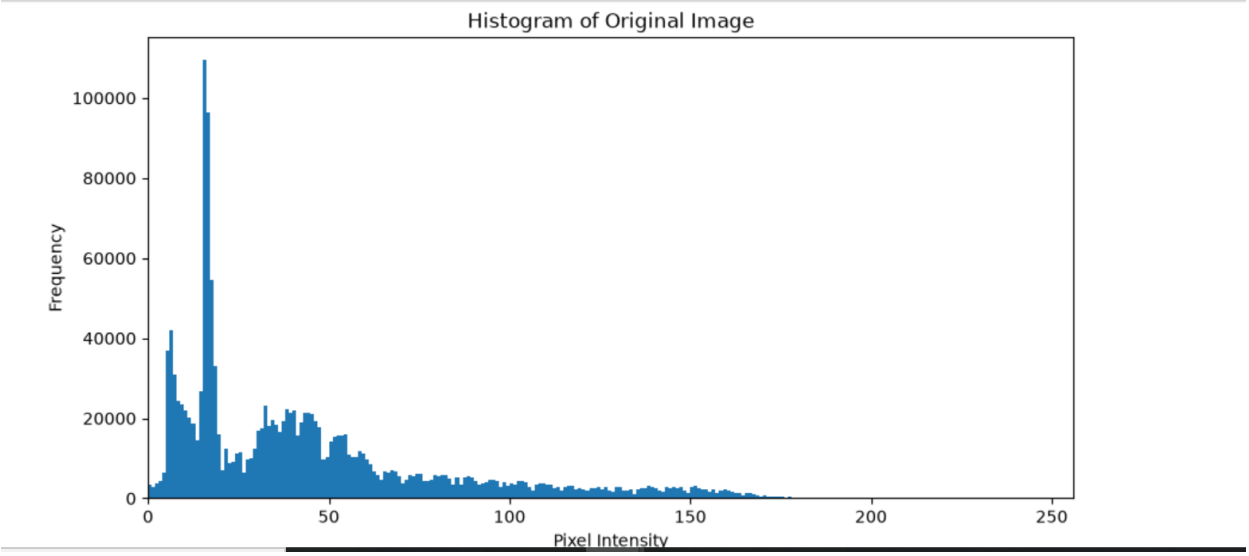

sample_hist = cv2.calcHist([sample], [0], None, [256], [0, 256]) print("Frequency at 0:",
int(sample_hist[0])) print("Frequency at 50:", int(sample_hist[50])) print("Frequency at
100:", int(sample_hist[100])) print("Frequency at 150:", int(sample_hist[150]))
print("Frequency at 200:", int(sample_hist[200])) print("Frequency at 255:",
int(sample_hist[255]))


hist = cv2.calcHist([img], [0], None, [256], [0, 256])


plt.figure(figsize=(8, 4)) plt.plot(hist)
plt.title("Histogram of Original Image")
plt.xlabel("Pixel Intensity") plt.ylabel("Frequency")
plt.xlim([0, 256]) plt.grid(True) plt.show()
```

Observation

The histogram shows the distribution of pixel intensities. A concentration on the left indicates more dark pixels, while a concentration on the right indicates more bright pixels.



11. Task 7 – Global Histogram Equalization

Objective: To improve global contrast by redistributing grayscale intensity values using the cumulative distribution function (CDF).

Concept: Histogram equalization uses the cumulative histogram to map old intensity values to new values. For the manual dry run, the mapping is s = 255 × (CDF − CDFmin)/(1 − CDFmin).

Dry Run / Manual Calculation

For pixels [50, 50, 100, 150], frequencies are 2, 1, 1 and probabilities are 0.5, 0.25, 0.25. CDF values are 0.5, 0.75, 1.0 and CDFmin = 0.5. The mapped values are 0, 127.5, 255; using nearest integer gives [0, 0, 128, 255].

Python Implementation

```
sample = np.array([50, 50, 100, 150], dtype=np.uint8)


values, counts = np.unique(sample, return_counts=True) probabilities =
counts / sample.size cdf = np.cumsum(probabilities) cdf_min = cdf.min()


mapped = np.rint(255 * (cdf - cdf_min) / (1 - cdf_min)).astype(np.uint8)


print("Unique Values:", values) print("Frequencies:", counts)
print("Probabilities:", probabilities) print("CDF:", cdf)
print("Equalized Values:", mapped)
  equalized_img = cv2.equalizeHist(img)


plt.figure(figsize=(12, 4))


plt.subplot(1, 2, 1); plt.imshow(img, cmap="gray"); plt.title("Original"); plt.axis("off") plt.subplot(1, 2, 2); plt.imshow(equalized_img,
cmap="gray"); plt.title("Global Histogram
Equalization"); plt.axis("off") plt.show()

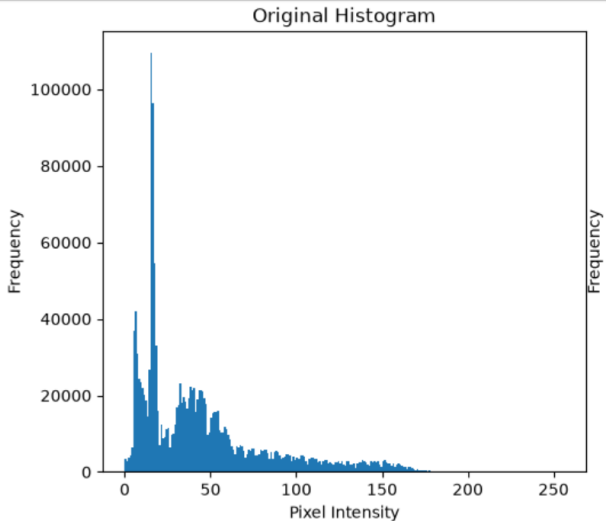

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1); plt.hist(img.ravel(), 256, [0, 256]); plt.title("Original Histogram") plt.subplot(1, 2, 2);
plt.hist(equalized_img.ravel(), 256, [0, 256]); plt.title("Equalized
Histogram") plt.show()
```

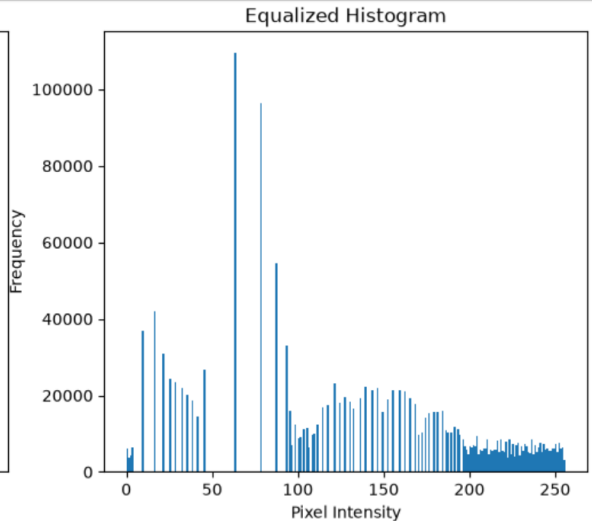
Observation

Global histogram equalization can improve overall contrast. It may also make some regions look too strong because the same global mapping is applied to the entire image.


Original Image


Histogram Equalized Image


Original Histogram
Frequency
Pixel Intensity
0
20000
40000
60000
80000
100000
0
50
100
150
200
250


Equalized Histogram
Frequency
Pixel Intensity
0
20000
40000
60000
80000
100000
0
50
100
150
200
250

12. Task 8 – CLAHE (Contrast Limited Adaptive Histogram Equalization)

Objective: To improve local contrast while limiting excessive amplification of noise.

Concept: CLAHE divides the image into small tiles, equalizes each tile, and limits histogram peaks using a clip limit. It is adaptive because local regions are processed separately. OpenCV combines the tiles smoothly.

Dry Run / Manual Calculation

Simplified dry run: histogram = [6, 2, 0, 0], clip cap = 4. The first bin has excess 2. Redistributing the excess equally across four bins adds 0.5 to each bin, giving [4.5, 2.5, 0.5, 0.5]. The cumulative values are [4.5, 7, 7.5, 8], and normalized CDF values are [0.5625, 0.875, 0.9375, 1.0]. This is a simplified illustration; OpenCV's clipLimit parameter is not the same as a literal cap of 4.

Python Implementation

```
# Simplified CLAHE dry-run illustration histogram = np.array([6, 2,
0, 0], dtype=float) clip_cap = 4


clipped = np.minimum(histogram, clip_cap) excess =
np.sum(histogram - clipped)
redistributed = clipped + excess / len(histogram)


cdf = np.cumsum(redistributed) normalized_cdf = cdf / cdf[-
1]
  print("Original Histogram:", histogram)
print("Clipped Histogram:", clipped) print("Excess:",
excess)
print("Redistributed Histogram:", redistributed) print("CDF:", cdf)
print("Normalized CDF:", normalized_cdf)


clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
clahe_img = clahe.apply(img)
equalized_img = cv2.equalizeHist(img)

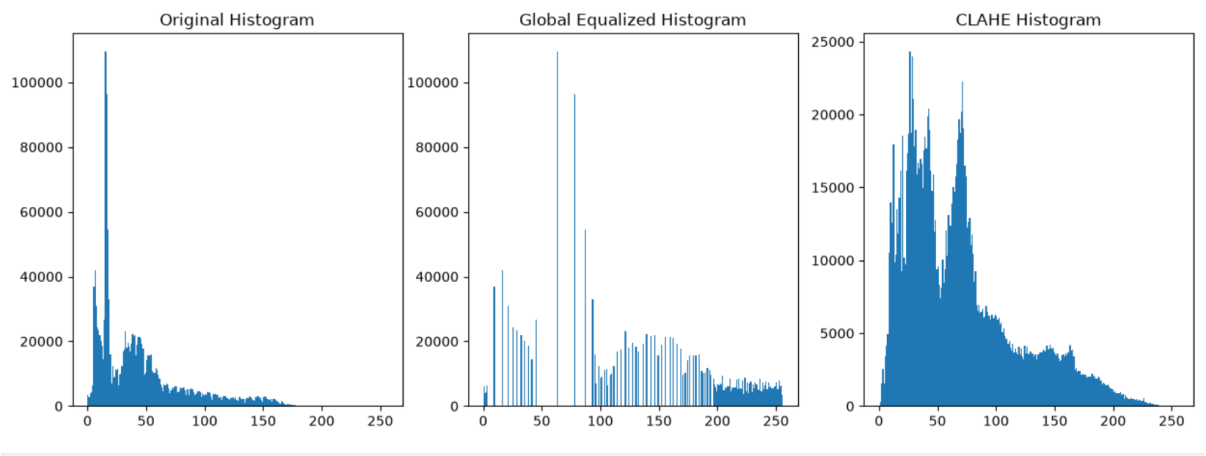

plt.figure(figsize=(15, 4))
plt.subplot(1, 3, 1); plt.imshow(img, cmap="gray"); plt.title("Original"); plt.axis("off") plt.subplot(1, 3, 2); plt.imshow(equalized_img,
cmap="gray"); plt.title("Global Equalization"); plt.axis("off")
plt.subplot(1, 3, 3); plt.imshow(clahe_img, cmap="gray"); plt.title("CLAHE"); plt.axis("off") plt.show()


plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1); plt.hist(img.ravel(), 256, [0, 256]); plt.title("Original Histogram") plt.subplot(1, 2, 2); plt.hist(clahe_img.ravel(),
256, [0, 256]); plt.title("CLAHE Histogram") plt.show()
```

Observation

CLAHE enhances local details and usually avoids some of the excessive contrast amplification that can occur with global equalization. Its result depends on clipLimit and tileGridSize.





```
Original Histogram: [6. 2. 0. 0.]
Clipped Histogram: [4. 2. 0. 0.]
Excess: 2.0
Redistributed Histogram: [4.5 2.5 0.5 0.5]
CDF: [4.5 7.  7.5 8. ]
Normalized CDF: [0.5625 0.875  0.9375 1.    ]
```

13. Task 9 – Fixed Binary Thresholding

Objective: To convert a grayscale image into a binary image using a fixed threshold value.

Concept: For binary thresholding, if r is greater than T, output becomes 255; otherwise, output becomes 0. In this experiment, T = 127.

Dry Run / Manual Calculation

For [0, 80, 127, 128, 180, 255] and T = 127, the result is [0, 0, 0, 255, 255, 255]. At T = 180, the result is [0, 0, 0, 0, 0, 255].

Python Implementation

```
pixels = np.array([0, 80, 127, 128, 180, 255], dtype=np.uint8)


threshold_value = 127
manual_result = np.where(pixels > threshold_value, 255, 0).astype(np.uint8)

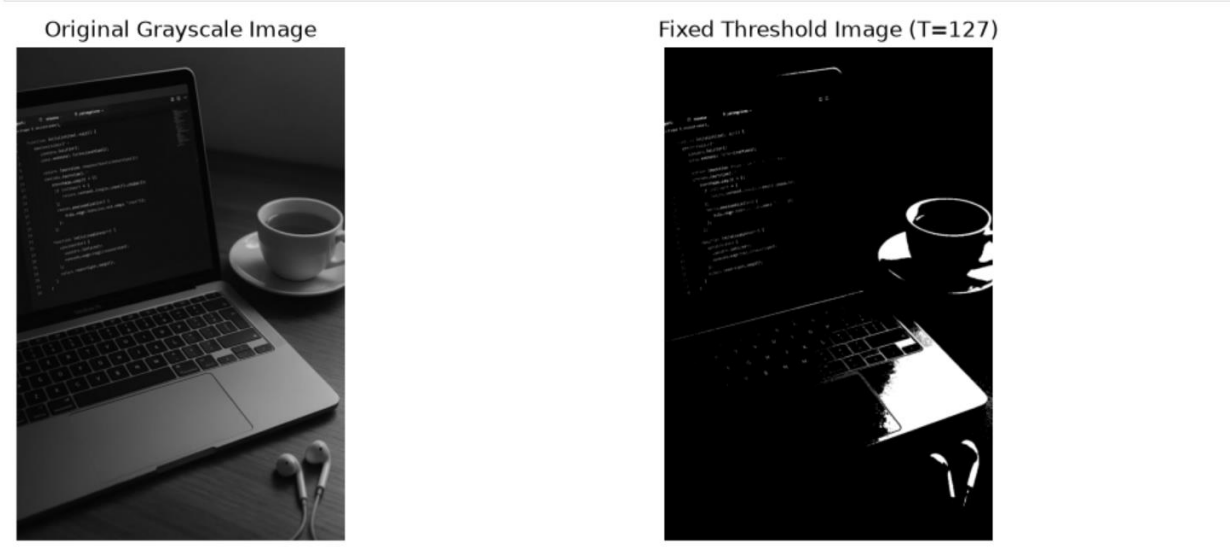

print("Original:", pixels)
print("Threshold 127:", manual_result)


ret, binary_img = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY) print("Threshold Used:", ret)
print("Unique Output Values:", np.unique(binary_img))


plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1); plt.imshow(img, cmap="gray"); plt.title("Original"); plt.axis("off") plt.subplot(1, 2, 2);
plt.imshow(binary_img, cmap="gray"); plt.title("Fixed Threshold T=127"); plt.axis("off") plt.show()
```

Observation

The grayscale image is converted into only two intensity levels: 0 (black) and 255 (white). The threshold determines which pixels become white.



```python
# Task 9: Fixed Binary Thresholding
pixels = np.array([0, 80, 127, 128, 180, 255])

T = 127

binary = np.where(pixels > T, 255, 0).astype(np.uint8)

print("Original Pixels:", pixels)
print("Threshold:", T)
print("Binary Output:", binary)
```

```
Original Pixels: [  0  80 127 128 180 255]
Threshold: 127
Binary Output: [  0   0   0 255 255 255]
```

14. Task 10 – Otsu's Thresholding

Objective: To automatically select a threshold value based on the image histogram.

Concept: Otsu's method tests possible threshold values and chooses the one that gives the best separation between two classes: background and foreground. It maximizes between-class variance.

Dry Run / Manual Calculation

For sample pixels [0, 1, 4, 5], candidate thresholds are tested. Thresholds 1, 2, and 3 give the same maximum between-class variance of 4. The smallest maximizing threshold is selected, so T = 1. The resulting binary array is [0, 0, 255, 255]

```
Threshold: 0
Group 0: [0]
Group 1: [1 4 5]
Weight 0: 0.25
Weight 1: 0.75
Mean 0: 0.0
Mean 1: 3.3333333333333335
Variance: 2.0833333333333335
----------------------
Threshold: 1
Group 0: [0 1]
Group 1: [4 5]
Weight 0: 0.5
Weight 1: 0.5
Mean 0: 0.5
Mean 1: 4.5
Variance: 4.0
----------------------
Threshold: 2
Group 0: [0 1]
Group 1: [4 5]
Weight 0: 0.5
Weight 1: 0.5
Mean 0: 0.5
Mean 1: 4.5
Variance: 4.0
----------------------
Threshold: 3
Group 0: [0 1]
Group 1: [4 5]
Weight 0: 0.5
Weight 1: 0.5
Mean 0: 0.5
Mean 1: 4.5
Variance: 4.0
----------------------
Threshold: 4
Group 0: [0 1 4]
Group 1: [5]
Weight 0: 0.75
Weight 1: 0.25
Mean 0: 1.6666666666666667
Mean 1: 5.0
Variance: 2.083333333333333
----------------------
Best Threshold: 1
Maximum Variance: 4.0
```



15. Task 11 – Adaptive Gaussian Thresholding

Objective: To create a binary image using a local threshold calculated from the neighborhood of each pixel.

Concept: Unlike fixed thresholding, adaptive thresholding calculates a different threshold for each local region. In Gaussian adaptive thresholding, nearby pixels contribute with Gaussian weights. The constant C is subtracted from the weighted local mean. A pixel becomes white if it is greater than the local threshold.

Dry Run / Manual Calculation

For the 3×3 sample [[100,100,100],[100,120,140],[140,140,140]], using Gaussian weights [[1,2,1],[2,4,2],[1,2,1]]/16 gives a weighted mean of 120 at the center. With C = 2, the local threshold is 118. Since 120 > 118, the center output is 255.

Python Implementation

```
sample = np.array([     [100, 100,
100],
    [100, 120, 140],
    [140, 140, 140]
], dtype=np.float32)
  weights = np.array([
    [1, 2, 1],
    [2, 4, 2],
    [1, 2, 1]
], dtype=np.float32) / 16


weighted_mean = np.sum(sample * weights) C = 2
local_threshold = weighted_mean - C center_pixel = sample[1, 1]
center_output = 255 if center_pixel > local_threshold else 0


print("Weighted Mean:", weighted_mean) print("C:", C)
print("Local Threshold:", local_threshold) print("Center Pixel:",
center_pixel) print("Center Output:", center_output)


adaptive_img = cv2.adaptiveThreshold(     img, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,     cv2.THRESH_BINARY,
    11, 2
)
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1); plt.imshow(img, cmap="gray"); plt.title("Original"); plt.axis("off") plt.subplot(1, 2, 2); plt.imshow(adaptive_img,
cmap="gray"); plt.title("Adaptive Gaussian
Threshold"); plt.axis("off") plt.show()
```

Observation

Adaptive Gaussian thresholding is useful when lighting varies across an image because the threshold is calculated locally rather than using one value for the whole image.




```
Weighted Mean: 120.0
C Value: 2
Threshold: 118.0
Center Pixel: 120
Final Output: 255
```

16. Overall Comparison

The experiments demonstrate that different enhancement methods serve different purposes. Brightness changes the overall intensity, contrast transformations change the spread of intensity values, and the negative operation reverses intensities. Gamma correction changes intensity nonlinearly. Histogram equalization improves global contrast, while CLAHE enhances local

contrast. Fixed thresholding uses a manually selected value, Otsu selects a threshold automatically, and adaptive Gaussian thresholding uses local neighborhood information.

17. Conclusion

In this lab, several image enhancement and intensity transformation techniques were implemented using Python, NumPy, OpenCV, and Matplotlib. The practical work showed how pixel values can be modified to improve brightness, contrast, and visibility of image details. Histogram methods were used to study and improve intensity distribution, while thresholding techniques converted grayscale images into binary images. The results also showed that no single method is best for every image; the appropriate technique depends on the image content and the desired result.